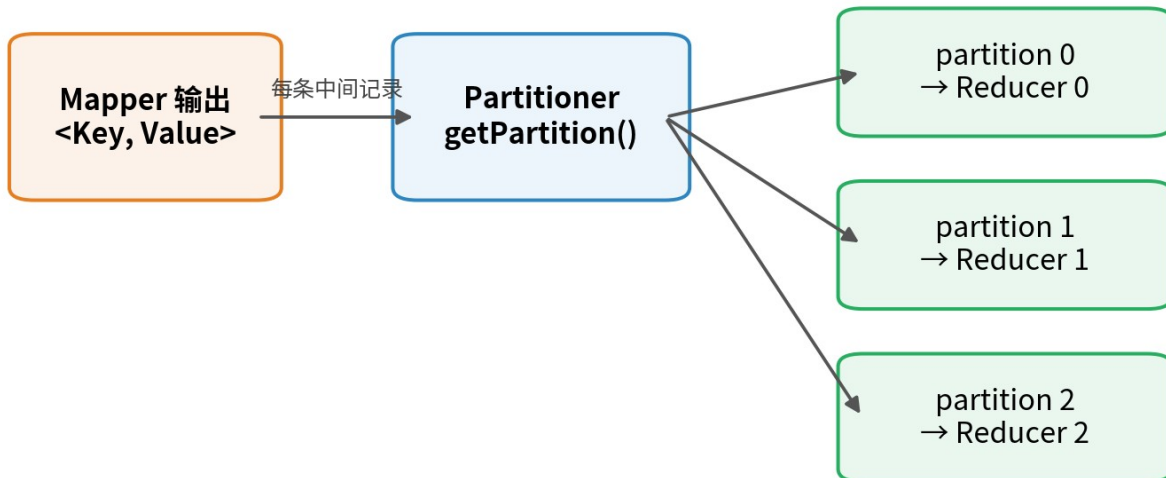


Hadoop MapReduce 原理详解

第八讲：Partitioner、Secondary Sort、全局排序与数据倾斜

从“把数据送到哪里”进阶到“数据以什么顺序到达 Reducer”



核心：Partitioner 不做聚合，它只决定“这条 Map 中间记录送到哪个 Reducer”。

本讲关键词：Partitioner • 多 Reducer • 复合 Key • Grouping Comparator • Total Order • Skew

一、本讲学习目标

- 理解 Partitioner 的职责：它决定 Map 中间记录进入哪个 Reducer，而不是负责排序或聚合。
- 看懂默认 HashPartitioner 的优点和局限，理解“同 Key 同 Reducer”与“负载均衡”是两个不同问题。
- 能够编写并注册一个简单的自定义 Partitioner，并知道哪些设计会破坏正确性。
- 理解 Secondary Sort 为什么必须同时设计复合 Key、Partitioner、Sort Comparator 和 Grouping Comparator。
- 理解多个 Reducer 只能保证“分区内有序”，以及 TotalOrderPartitioner 如何构造真正的全局排序。
- 理解热点 Key/数据倾斜为什么会造成长尾，并掌握加盐 + 两阶段聚合这一通用解决模式。

先抓住主线：前七讲解决的是“MapReduce 如何跑起来”；本讲开始解决“数据如何被有意识地组织”。如果把 Shuffle 看成一条物流网络，Partitioner 就是分拣规则，Sort 是每个目的地内部的排队规则，Grouping 决定 Reducer 一次处理哪一组。

二、Partitioner 到底在计算什么？

Mapper 产生一条中间记录后，Hadoop 必须回答一个非常具体的问题：这条记录应该送给 Reducer 0、Reducer 1，还是 Reducer 2？这个决策就是 Partitioner 的工作。

Partitioner 的核心接口（概念化）

```
public abstract class Partitioner<KEY, VALUE> {
    public abstract int getPartition(
        KEY key, VALUE value, int numReduceTasks);
}
```

输入是中间 Key、Value 和 Reducer 数量；输出是一个分区编号，必须落在 0 到 numReduceTasks-1 之间。Hadoop 官方 API 明确指出：分区总数等于 Reduce Task 的数量，因此 Partitioner 实际上控制了中间 Key 最终送往哪个 Reducer。

组件	它回答的问题	典型输出
Mapper	这条原始记录要产生什么中间数据？	<商品 A, 1>
Partitioner	这条中间记录送哪个 Reducer？	partition 2
Sort	这个 Reducer 内部按什么顺序排列 Key？	A < B < C
Grouping	哪些相邻 Key 算同一组，进入同一次 reduce()？	同一 user
Reducer	这一组 Value 最终怎么聚合？	sum / max / join

关键边界：当只有 1 个 Reducer 时，所有中间记录天然只有一个去向，所以 Partitioner 没有“选择空间”。官方 API 也说明 Partitioner 只在存在多个 Reducer 时创建。

三、默认 HashPartitioner：为什么通常够用？

Hadoop 默认使用 HashPartitioner。它基于 Key 的 hashCode() 把 Key 映射到某个分区。可以把它抽象理解成：

$$\text{partition} \approx \text{hash}(\text{key}) \bmod \text{numReduceTasks}$$

真正重要的不是背公式，而是理解两个性质。

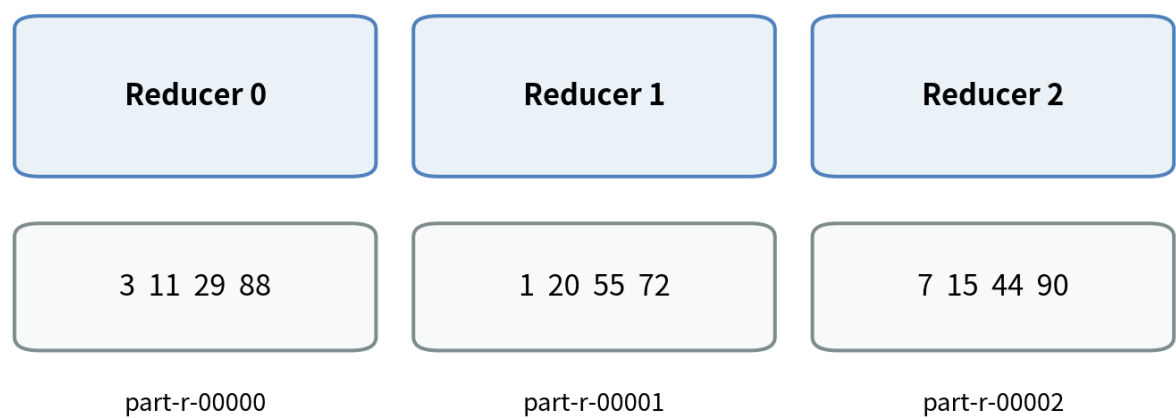
- 1. 同一个 Key 的哈希结果稳定时，会被映射到同一个分区，因此同一个 Key 的所有记录能在同一个 Reducer 中汇总。
- 2. 不同 Key 会被“尽量打散”到多个 Reducer，但这不等于数据量一定均衡，因为 Key 的出现频率可能极不均匀。

中间数据	Key 数量	频率分布	HashPartitioner 可能结果
A/B/C/D 各 100 万条	4	均匀	通常较均衡
北京 1000 万，其余城市各 10 万	很多	严重偏斜	北京所在 Reducer 仍会很重
用户 ID 基本均匀	数百万	较均匀	通常很适合 Hash

重要结论：“哈希分得散”只能缓解 Key 数量层面的分布；如果某一个 Key 自己就占了绝大多数数据，哈希无法把这个 Key 拆给多个 Reducer，因为普通 Reduce 语义要求同一 Key 的记录必须聚到一起。

四、多个 Reducer 为什么不等于全局有序？

多个 Reducer：每个分区内部有序 ≠ 全局有序



HashPartitioner 只保证同一个 Key 去同一个 Reducer；它不会按 Key 大小划分范围。

图 1 默认哈希分区下，每个 Reducer 内部排序，但不同 part 文件之间不存在 Key 范围关系

前面学过：Reducer 的输入会按 Key 排序。因此很多初学者会自然推断“最终输出就是全局排序的”。问题在于：排序发生在每个 Reducer 自己的分区内部。默认 HashPartitioner 并不是按 Key 大小划分区间。

因此 3 个 Reducer 常见输出是 part-r-00000、part-r-00001、part-r-00002。每个文件内部可以按 Key 有序，但把 00000、00001、00002 简单拼起来，并不能保证整体有序。

一句话：多 Reducer + 默认 Hash = 分区内有序；要全局有序，必须让“分区边界本身也按 Key 范围有序”。

五、什么时候需要自定义 Partitioner?

当“Key 应该去哪台 Reducer”本身带有业务含义，或者默认 Hash 无法满足后续处理约束时，就需要自定义 Partitioner。典型场景包括：

- 按省份/地区固定分区，方便生成独立结果文件；
- 按数值区间分区，例如成绩 0-59、60-79、80-100；
- 复合 Key 做 Secondary Sort 时，只用“主 Key”决定 Reducer；
- 为某些特殊 Key 做定制路由（但必须谨慎避免倾斜）。

六、代码例子：按成绩区间分到 3 个 Reducer

假设 Mapper 输出 <score, studentName>，我们希望不按哈希随机分配，而是强制：不及格 → Reducer 0，60-79 → Reducer 1，80-100 → Reducer 2。

自定义 ScorePartitioner（核心代码）

```
public static class ScorePartitioner
    extends Partitioner<IntWritable, Text> {

    @Override
    public int getPartition(IntWritable key, Text value,
                           int numReduceTasks) {
        int score = key.get();    // Mapper 输出的成绩
        if (score < 60) return 0; // 不及格 → Reducer 0
        if (score < 80) return 1; // 60~79 → Reducer 1
        return 2;                // 80~100 → Reducer 2
    }
}
```

Driver 中的关键配置

```
// Driver 中注册自定义 Partitioner。
job.setPartitionerClass(ScorePartitioner.class);

// 这个分区规则写死了 0、1、2，因此这里必须有 3 个 Reducer。
job.setNumReduceTasks(3);
```

隐藏风险：如果把 Reducer 数从 3 改成 2，而 Partitioner 仍可能返回 2，就会产生非法分区。更健壮的代码应让分区规则与 numReduceTasks 协同，或者显式检查配置。

七、正确性底线：Grouping 与 Partitioner 必须“相容”

这是高级 MapReduce 中最容易被忽略、但最关键的一条规则：如果两条中间记录在 Reduce 阶段应该被视为同一组，那么它们必须先被 Partitioner 送到同一个 Reducer。

记录 1	记录 2	Grouping 认为同组?	Partitioner 是否必须同分区?
(A, 09:00)	(A, 10:00)	是	必须
(A, 09:00)	(B, 09:00)	否	不要求

如果 Grouping Comparator 只看 user=A，但 Partitioner 却使用整个复合 Key (user,time) 的哈希，那么 A 的 09:00 和 A 的 10:00 可能被送到不同 Reducer。此时它们永远不可能在同一次 reduce() 中相遇，逻辑直接错误。

可记忆规则：“先分区，后排序分组。” 因此，Grouping 定义的同一逻辑组，必须在更早的 Partition 阶段就被保证送到同一 Reducer。

八、Secondary Sort 到底要解决什么？

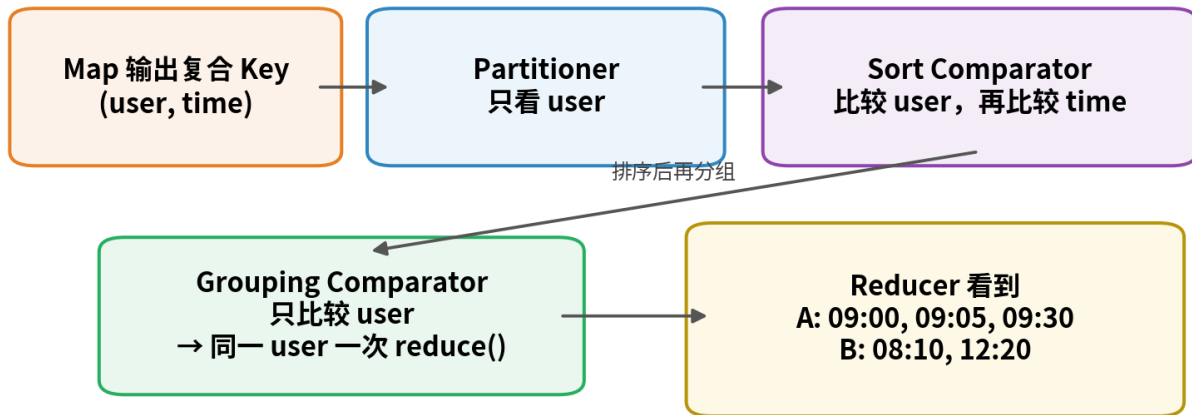
普通 MapReduce 很擅长“按 Key 分组”，但现实中经常还有第二层要求：同一个组内部也必须按照另一个字段排序。

例如网站行为日志：

原始记录（顺序是乱的）

```
A, 09:30, /pay
B, 12:20, /index
A, 09:00, /index
A, 09:05, /product
B, 08:10, /login
```

目标不是简单得到 A 的所有行为，而是让 Reducer 对 A 的一次调用中，行为天然按时间排列：09:00 → 09:05 → 09:30。这个模式就是 Secondary Sort（二次排序）。

Secondary Sort：按“用户”分组，但组内按“时间”排序

三件套必须协同：同组去同一 Reducer + 全复合键排序 + 只按主键分组。

图 2 Secondary Sort 的三个协同规则

九、Secondary Sort 的复合 Key：为什么不能只用 user?

如果 Mapper 只输出 key=user，那么 Hadoop 只知道按 user 排序；时间 timestamp 在 Value 里，框架不会自动拿 Value 做排序。解决办法是把“主字段 + 次字段”一起提升到 Key：

Map 输出 Key = (user, timestamp) Map 输出 Value = 真实业务数据（例如 URL）

于是完整 Key 可以排序：先 user，再 timestamp。与此同时，我们又不希望每个 (user,timestamp) 都单独调用 reduce()，所以还需要自定义 Grouping Comparator，让分组时只看 user。

十、复合 Key 的 WritableComparable 示例

复合 Key 的核心逻辑（示意代码）

```
public class UserTimeKey implements WritableComparable<UserTimeKey> {
    private Text user = new Text();
    private LongWritable time = new LongWritable();

    @Override
    public void write(DataOutput out) throws IOException {
        user.write(out);      // Hadoop 网络/磁盘传输时序列化 user
        time.write(out);      // 再序列化 time
    }

    @Override
    public void readFields(DataInput in) throws IOException {
        user.readFields(in);  // 反序列化时必须与 write() 顺序一致
        time.readFields(in);
    }

    @Override
    public int compareTo(UserTimeKey other) {
        // 第一排序字段: user
        int c = user.compareTo(other.user);
        if (c != 0) return c;

        // user 相同时, 第二排序字段: time
        return time.compareTo(other.time);
    }

    // 省略构造器、getter/setter、hashCode/equals 等样板代码。
}
```

底层含义：compareTo() 定义的是“排序语义”。整个复合 Key 排序后，同一 user 的记录会相邻；同一 user 内部又按 time 排好。

十一、Secondary Sort 的 Partitioner：只按主 Key 分区

Partitioner：只看 user，不看 time

```
public class UserPartitioner
    extends Partitioner<UserTimeKey, Text> {

    @Override
    public int getPartition(UserTimeKey key,
                           Text value,
                           int numReduceTasks) {

        // 只根据 user 分区。
        // 这样同一个 user 的所有时间点一定进入同一个 Reducer。
        return (key.getUser().hashCode() & Integer.MAX_VALUE)
            % numReduceTasks;
    }
}
```

这里有一个很重要的结构：排序使用完整 Key；分区只使用主 Key。这两个规则看起来“故意不一样”，正是 Secondary Sort 能工作的原因。

十二、Grouping Comparator：让同一 user 只调用一次 reduce()

Grouping Comparator：只看 user

```
public class UserGroupingComparator extends WritableComparator {
    public UserGroupingComparator() {
        super(UserTimeKey.class, true);
    }

    @Override
    public int compare(WritableComparable a,
                      WritableComparable b) {
        UserTimeKey k1 = (UserTimeKey) a;
        UserTimeKey k2 = (UserTimeKey) b;

        // 分组时忽略 time，只比较 user。
        return k1.getUser().compareTo(k2.getUser());
    }
}
```

Driver 关键配置

```
// Driver 中组合这几件事
job.setPartitionerClass(UserPartitioner.class);
job.setGroupingComparatorClass(UserGroupingComparator.class);

// 如果 UserTimeKey.compareTo() 已经定义了 user→time 的排序，
// 可以直接使用它的自然排序；也可以显式设置 Sort Comparator。
```

最终效果：Partitioner 保证同一 user 去同一个 Reducer；Sort 保证同一 user 内按 time 排序；Grouping 保证同一 user 的记录在一次 reduce() 中处理。

进一步精确：如果多个记录拥有完全相同的 user 和 time，而你还要求它们之间有确定顺序，就需要再加入第三个 tie-breaker 字段，例如事件 ID。否则“同一复合 Key 内部”的相对顺序不应被假定。

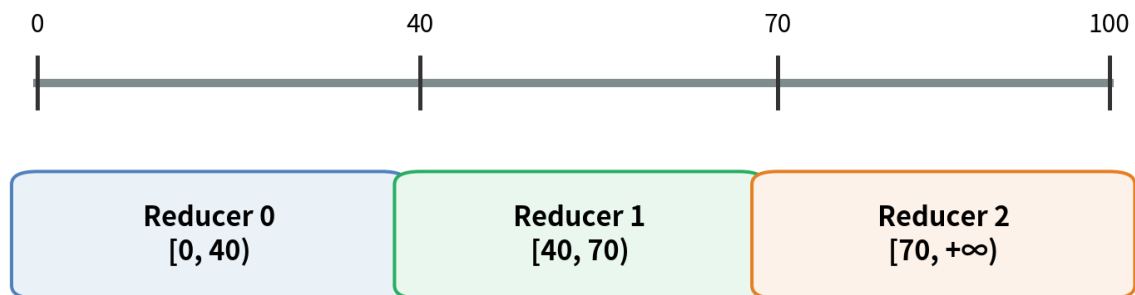
十三、全局排序：为什么一个 Reducer 很简单，但不一定划算？

如果把 Reducer 数量设为 1，所有 Key 都进入同一个 Reducer，该 Reducer 内部的 Sort 自然可以得到全局有序输出。这个办法概念最简单，但失去了 Reduce 端并行性；数据很大时会形成单点瓶颈。

方案	全局有序	Reduce 并行	主要问题
1 个 Reducer	是	无	规模大时慢、单点瓶颈
多个 Reducer + HashPartitioner	否	有	各 part 内有序但整体无序
多个 Reducer + TotalOrderPartitioner	是	有	需要采样和范围边界

十四、TotalOrderPartitioner：范围分区构造全局有序

TotalOrderPartitioner：用“范围”而不是“哈希”分区



3 个 Reducer 需要 2 个边界点 (R-1)。边界通常通过抽样估计，再写入 partition file。

各 part 文件内部排序 + 分区范围有序 → 按分区编号连接后形成全局顺序。

图 3 范围分区让 Reducer 0 的所有 Key 都小于 Reducer 1 的 Key

TotalOrderPartitioner 的核心不是复杂公式，而是“边界点”。假设有 R 个 Reducer，就需要 R-1 个有序边界，把 Key 空间切成 R 个连续范围。Hadoop 提供 InputSampler，可通过输入抽样生成 partition file，供 TotalOrderPartitioner 读取。

```
// 思想示例：3 个 Reducer，边界为 40 和 70
Reducer 0: key < 40
Reducer 1: 40 <= key < 70
Reducer 2: key >= 70
```

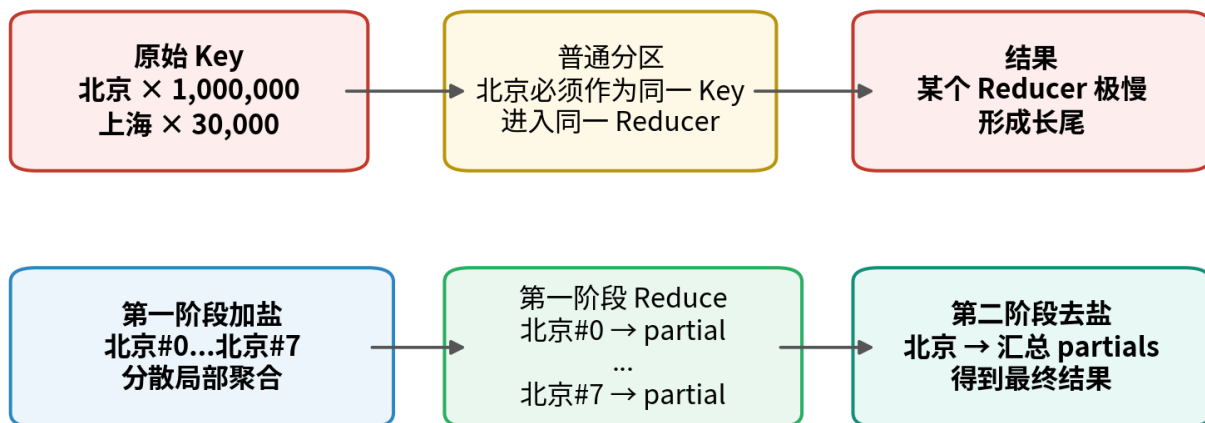
因为每个 Reducer 内部仍会排序，同时 Reducer 0 的最大 Key 不会超过 Reducer 1 的最小 Key，所以按分区编号读取 part 文件时，就能得到全局顺序。

为什么要采样：如果边界随便取，虽然仍可能保持“范围有序”，但负载可能极不均衡。例如 99% 的 Key 都小于 40，那么 Reducer 0 会被压垮。采样的目的之一就是估计分布，选择更合理的分位点。

十五、数据倾斜：真正的问题不是 Reducer 少，而是工作量不均

一个 Job 有 20 个 Reducer，并不意味着工作自动平均分成 20 份。最终完成时间由最慢的那一个 Task 决定，因此只要一个 Reducer 特别重，就会出现前面讲过的“99% 卡住”。

热点 Key：为什么“加 Reducer”也可能没用



关键：普通 Reduce 的正确性要求“同一逻辑 Key 汇总到一起”；要拆热点 Key，通常必须改成两阶段算法。

图 4 热点 Key 与“两阶段加盐聚合”

典型倾斜来源：

- 热点 Key：某一个 Key 本身出现次数巨大；
- 分区函数不合理：大量不同 Key 被路由到同一个分区；
- Join 倾斜：某个关联键一侧或两侧的数据量异常大；
- 业务分布天然偏斜：例如“其他/未知”类别占据绝大多数记录。

十六、为什么“再加几个 Reducer”不一定能解决热点 Key?

假设北京这个 Key 有 1000 万条。即使你从 10 个 Reducer 增加到 100 个 Reducer，只要仍使用普通 Reduce 语义，北京的 1000 万条记录必须落到同一个逻辑组，也就必须由同一个 Reducer 完成最终聚合。

本质：Reducer 数量增加，只能增加“不同分区之间”的并行度；它不能自动把“同一个逻辑 Key”拆成多个独立最终聚合。

十七、通用解法：加盐 + 两阶段聚合

对于 COUNT、SUM 这类可以先做局部聚合、再合并的操作，可以主动把热点 Key 暂时拆成多个“伪 Key”。

```
// 第一阶段 Map：给热点 Key 加一个小范围 salt
北京 -> 北京#0
北京 -> 北京#3
北京 -> 北京#7
...

// 第一阶段 Reduce：分别得到局部结果
北京#0 -> 1,250,000
北京#1 -> 1,247,000
...

// 第二阶段 Map：去掉 salt
北京#0 -> 北京, 1,250,000
北京#1 -> 北京, 1,247,000

// 第二阶段 Reduce：再次求和
北京 -> 10,000,000
```

这就是典型的“先打散热点，再局部聚合，最后归并”。它实际上把一个原本不能并行的热点 Key，改写成多个可并行的 partial aggregate。

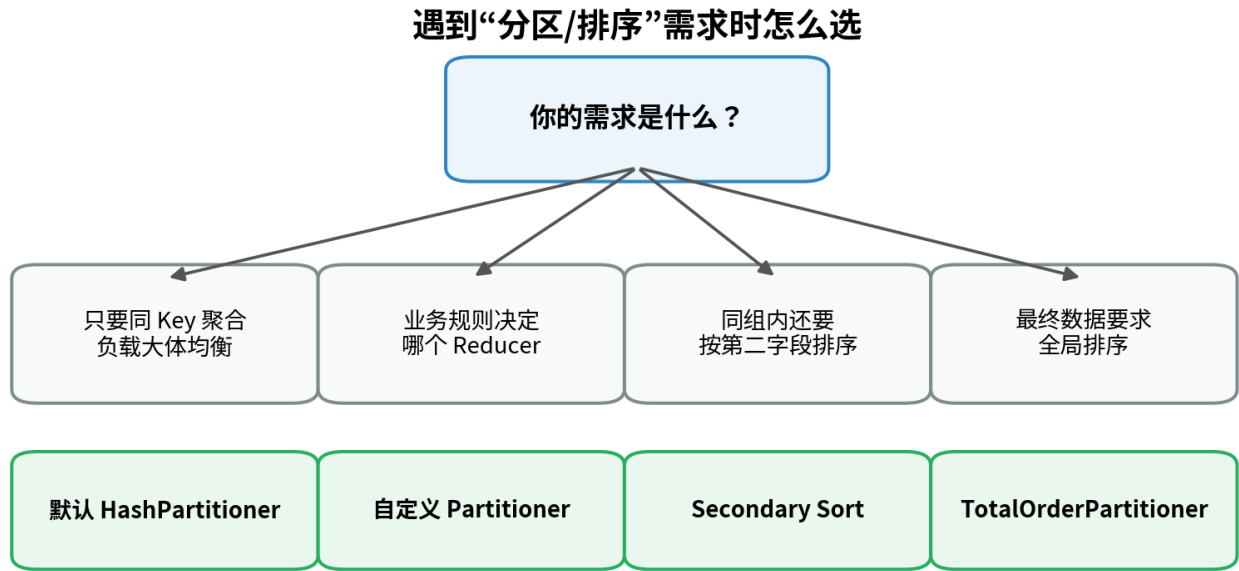
不是所有计算都能直接这么做：SUM/COUNT/MAX/MIN 很自然；平均值不能简单“平均各分区平均值”，应携带 (sum,count) 再合并。中位数、复杂去重、非结合性操作需要重新设计状态，不能机械套用。

十八、自定义 Partitioner 能否直接解决热点 Key?

如果你让同一个 Key 随机进入多个 Reducer，确实能打散负载，但会破坏普通 Reduce 的语义：同一个 Key 的 Value 被分散后，没有任何一个 Reducer 能看到完整集合。

所以“随机 Partitioner 把北京拆开”不是完整方案；它必须和第二阶段重新聚合配套。也就是说，解决热点 Key 的关键不只是改路由，而是改算法结构。

十九、把五个概念放在一起：决策表



若问题是热点 Key 造成倾斜：先诊断分布，再考虑加盐、两阶段聚合或业务定制。

图 5 分区、排序与倾斜问题的选择框架

需求	首选机制	一句话理解
普通分组聚合	HashPartitioner	同 Key 稳定去同分区
业务指定路由	自定义 Partitioner	自己定义“送哪个 Reducer”
组内第二字段排序	Secondary Sort	复合 Key 排序 + 主 Key 分组
多个 Reducer 仍要求全局排序	TotalOrderPartitioner	按范围切分 Key 空间
热点 Key 导致倾斜	加盐 + 两阶段聚合	先拆热点，再合并 partial

二十、上机观察：3 个 Reducer 到底生成什么？

你可以直接在现有 Hadoop 3.3.5 虚拟机中观察。先按课程统一规则启动服务并检查健康状态：

```
# 1. 启动 HDFS 和 YARN
start-dfs.sh
start-yarn.sh

# 2. 检查 Java 进程
jps

# 预期至少能看到 NameNode、DataNode、ResourceManager、NodeManager 等。
```

然后在已有 WordCount Driver 中临时加入：

```
// 强制使用 3 个 Reducer，观察输出文件数量。
job.setNumReduceTasks(3);
```

```
# 清理旧输出目录 (Hadoop 输出目录必须事先不存在)
hdfs dfs -rm -r -f /mr/output_multi

# 提交你的 WordCount Job
hadoop jar wordcount.jar WordCount /mr/input /mr/output_multi

# 查看输出目录
hdfs dfs -ls /mr/output_multi

# 分别查看 3 个 Reducer 结果
hdfs dfs -cat /mr/output_multi/part-r-00000
hdfs dfs -cat /mr/output_multi/part-r-00001
hdfs dfs -cat /mr/output_multi/part-r-00002
```

观察重点不是“有 3 个文件”这么简单，而是验证：相同单词只会出现在一个 part 文件中；每个文件内部的 Key 有序；但三个文件之间不能假设是全局 Key 顺序。

二十一、初学者最容易混淆的 12 个问题

- 1. Partitioner 是在 Mapper 之前还是之后？** 之后。Mapper 先产生中间 Key-Value；随后 Map 端根据 Partitioner 决定每条记录所属 Reduce 分区。
- 2. 一个 partition 是否一定对应一台物理机器？** 不是。partition 对应逻辑 Reducer/Reduce Task；YARN 再决定该 Task 运行在哪个 Container/节点。
- 3. 分区数为什么通常等于 Reducer 数？** 因为每个 Reducer 负责一个中间输出分区。Partitioner 的返回值本质就是目标 Reduce 分区编号。
- 4. 相同 Key 能否进入不同 Reducer？** 普通 MapReduce 聚合语义下不应如此，否则一次 reduce() 无法看到这个 Key 的完整 Value 集。
- 5. HashPartitioner 能保证绝对均衡吗？** 不能。它能打散很多不同 Key，但无法消除热点 Key 自身的频率偏斜。
- 6. Secondary Sort 是对 Value 排序吗？** 效果上常表现为“组内 Value 有序”，但框架真正排序的是扩展后的复合 Key。
- 7. Grouping Comparator 会改变排序吗？** 不会。排序由 Key/Sort Comparator 决定；Grouping 只决定哪些相邻排序 Key 合并成一次 reduce() 调用。
- 8. Secondary Sort 为什么要自定义 Partitioner？** 因为复合 Key 的完整哈希可能让同一主 Key 的不同次 Key 跑到不同 Reducer，必须只按主 Key 分区。
- 9. 多 Reducer 输出为什么不是全局有序？** 因为默认 Hash 分区没有 Key 范围关系，每个 Reducer 只是各自内部排序。
- 10. 全局排序是不是只能使用 1 个 Reducer？** 不是。TotalOrderPartitioner 可以用范围分区保持多个 Reducer 并行，同时形成全局顺序。
- 11. 多加 Reducer 能解决所有倾斜吗？** 不能。热点 Key 仍会落到单个逻辑组；需要改算法结构，例如加盐两阶段聚合。

12. 加盐是不是改变最终 Key? 只是第一阶段临时改变。第二阶段会去掉 salt，把 partial 重新按原始逻辑 Key 汇总。

二十二、迁移到 Spark：这些思想并没有消失

以后学习 Spark 时，你会再次看到几乎同一组思想，只是 API 和执行引擎不同。

MapReduce 概念	Spark 中常见对应思想
Partitioner / Reduce 分区	partitionBy / shuffle partitions / repartition
按 Key Shuffle	groupByKey / reduceByKey 等宽依赖 Shuffle
Secondary Sort	复合 Key + 分区 + sortWithinPartitions 等组合
Total Order / Range Partition	RangePartitioner / repartitionByRange (不同 API 场景)
数据倾斜	salting、预聚合、AQE/Skew Join 等现代机制

Bridging：跨框架底层结构：无论 Hadoop、Spark 还是分布式 SQL，都绕不开三个问题：数据怎么分区？什么时候必须 Shuffle？如何避免某个分区成为长尾？API 会变，但这三个问题长期存在。

二十三、本讲最终心智模型

把本讲压缩成一条完整链条：

```

Mapper 输出 <K,V>
    ↓
Partitioner: 决定去哪个 Reducer
    ↓
每个 Reducer 内 Sort: 决定 Key 的顺序
    ↓
Grouping: 决定哪些相邻 Key 属于同一次 reduce()
    ↓
Reducer: 对一个逻辑组完成最终计算
  
```

再记住四句：

- 默认 HashPartitioner 解决“普通分散”，不保证热点 Key 均衡。
- Secondary Sort 的本质是“分区规则、排序规则、分组规则三者协同”。
- 全局排序需要“分区之间也有范围顺序”，TotalOrderPartitioner 就是在做这件事。
- 数据倾斜是算法结构问题时，不能只调参数；热点 Key 往往需要两阶段改写。

二十四、练习：先自己推演，再看答案

- 有 4 个 Reducer，默认 HashPartitioner。某个 Key=X 有 1 亿条记录。把 Reducer 增加到 20 个后，X 能否自动分散到 20 个 Reducer？为什么？
- 需要“每个学生的成绩按课程时间排序输出”，主分组字段应该是什么？复合 Key 应包含哪些字段？Partitioner、Grouping 分别看哪些字段？
- 有 5 个 Reducer，要使用 TotalOrderPartitioner，需要多少个分区边界点？为什么？
Partitioner、Secondary Sort、全局排序与数据倾斜

6. 计算城市平均消费，热点城市需要加盐。第一阶段不能只输出局部平均值，应该输出什么状态，第二阶段怎么合并？

参考答案：

- 1) 不能。相同逻辑 Key 在普通 Reduce 语义下仍应进入同一 Reducer；要拆开必须改变算法，例如加盐 + 两阶段聚合。
- 2) 主分组字段是学生 ID；复合 Key 可用 (studentId, courseTime)。Partitioner 只看 studentId；排序看 studentId→courseTime；Grouping 只看 studentId。
- 3) 4 个边界点。R 个连续范围只需要 R-1 个切分边界。
- 4) 输出 (局部 sum, 局部 count)，第二阶段分别求总 sum 和总 count，最终 $average = totalSum / totalCount$ 。

参考与核对资料

本讲 API 语义按 Apache Hadoop 3.3.5 官方文档核对：

- org.apache.hadoop.mapreduce.Partitioner：Partitioner 控制 Map 中间 Key 的分区，分区数量与 Reduce Task 数一致。
- org.apache.hadoop.mapreduce.lib.partition.HashPartitioner：基于 Object.hashCode() 对 Key 分区。
- org.apache.hadoop.mapreduce.Reducer：官方 API 对 Secondary Sort 给出“扩展 Key + Grouping Comparator”的标准模式。
- org.apache.hadoop.mapreduce.lib.partition.TotalOrderPartitioner 与 InputSampler：通过外部分区边界文件实现总序分区，InputSampler 用于抽样并生成分区文件。

官方文档入口：<https://hadoop.apache.org/docs/r3.3.5/api/>